A Technical Introduction to Softmax function

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Section 1

where the action value q t (a) {\displaystyle q_{t}(a)} corresponds to the expected reward of following action a and τ {\displaystyle \tau } is called a temperature parameter (in allusion to statistical mechanics). For high temperatures (τ → ∞ {\displaystyle \tau \to \infty }), all actions have nearly the same probability and the lower the temperature, the more expected rewards affect the probability. For a low temperature (τ → 0 + {\displaystyle \tau \to 0^{+}}), the probability of the action with the highest expected reward tends to 1.

Section 2

∂ ∂ q k σ (q , i) = σ (q , k) (δ i k − σ (q , i)) . {\displaystyle {\frac {\partial }{\partial q_{k}}}\sigma ({\textbf {q}},i)=\sigma ({\textbf {q}},k)(\delta _{ik}-\sigma ({\textbf {q}},i)).}

Section 3

Alternatives The softmax function generates probability predictions densely distributed over its support. Other functions like sparsemax or α-entmax can be used when sparse probability predictions are desired. Also the Gumbel-softmax reparametrization trick can be used when sampling from a discrete-discrete distribution needs to be mimicked in a differentiable manner.

Section 4

Gradients The softmax function is also the gradient of the LogSumExp function: ∂ ∂ z i LSE ■ (z) = exp ■ z i ∑ j = 1 K exp ■ z j = σ (z) i , for i = 1 , … , K , z = (z 1 , … , z K) ∈ R K , {\displaystyle {\frac {\partial }{\partial z_{i}}}\operatorname {LSE} (\mathbf {z})={\frac {\exp z_{i}}{\sum _{j=1}^{K}\exp z_{j}}}=\sigma (\mathbf {z})_{i},\quad {\text{ for }}i=1,\dotsc ,K,\quad \mathbf {z} =(z_{1},\,\dotsc ,\,z_{K})\in \mathbb {R} ^{K},} where the LogSumExp function is defined as LSE ■ (z 1 , … , z n) = log ■ (exp ■ (z 1) + ■ + exp ■ (z n)) {\displaystyle \operatorname {LSE} (z_{1},\,\dots ,\,z_{n})=\log \left(\exp(z_{1})+\cdots +\exp(z_{n})\right)} . The gradient of softmax is thus ∂ z j σ i = σ i (δ i j − σ j) {\displaystyle \partial _{z_{j}}\sigma _{i}=\sigma _{i}(\delta _{ij}-\sigma _{j})} .

Section 5

Example With an input of (1, 2, 3, 4, 1, 2, 3), the softmax is approximately (0.024, 0.064, 0.175, 0.475, 0.024, 0.064, 0.175). The output has most of its weight where the "4" was in the original input. This is what the function is normally used for: to highlight the largest values and suppress values which are significantly below the maximum value. But note: a change of temperature changes the output. When the temperature is multiplied by 10, the inputs are effectively (0.1, 0.2, 0.3, 0.4, 0.1, 0.2, 0.3) and the softmax is approximately (0.125, 0.138, 0.153, 0.169, 0.125, 0.138, 0.153). This shows that high temperatures de-emphasize the maximum value. Computation of this example using Python code:

Section 6

For any input, the outputs must all be positive and they must sum to unity. ... Given a set of unconstrained values, ■ V j (x) {\displaystyle V_{j}(x)} ■, we can ensure both conditions by using a Normalised Exponential transformation:

Section 7

Geometrically, softmax is constant along diagonals: this is the dimension that is eliminated, and corresponds to the softmax output being independent of a translation in the input scores (a choice of 0 score). One can normalize input scores by assuming that the sum is zero (subtract the average: c {\displaystyle \mathbf {c} } where c = 1 n ∑ z i {\textstyle c={\frac {1}{n}}\sum z_{i}}), and then the softmax takes the hyperplane of points that sum to zero, ∑ z i = 0 {\textstyle \sum z_{i}=0} , to the open simplex of positive values that sum to 1 ∑ σ (z) i = 1 {\textstyle \sum \sigma (\mathbf {z})_{i}=1} , analogously to how the exponent takes 0 to 1, e 0 = 1 {\displaystyle e^{0}=1} and is positive. By contrast, softmax is not invariant under scaling. For instance, σ ((0 , 1)) = (1 / (1 + e) , e / (1 + e)) {\displaystyle \sigma {\bigl (}(0,\,1){\bigr)}={\bigl (}1/(1+e),\,e/(1+e){\bigr)}} but σ ((0 , 2)) = (1 / (1 + e 2) , e 2 / (1 + e 2)) . {\displaystyle \sigma {\bigl (}(0,2){\bigr)}={\bigl (}1/\left(1+e^{2}\right),\,e^{2}/\left(1+e^{2}\right){\bigr)}.}

Section 8

This can be seen as the composition of K linear functions x ■ x T w 1 , … , x ■ x T w K {\displaystyle \mathbf {x} \mapsto \mathbf {x} ^{\mathsf {T}}\mathbf {w} _{1},\ldots ,\mathbf {x} \mapsto \mathbf {x} ^{\mathsf {T}}\mathbf {w} _{K}} and the softmax function (where x T w {\displaystyle \mathbf {x} ^{\mathsf {T}}\mathbf {w} } denotes the inner product of x {\displaystyle \mathbf {x} } and w {\displaystyle \mathbf {w} }). The operation is equivalent to applying a linear operator defined by w {\displaystyle \mathbf {w} } to tuples x {\displaystyle \mathbf {x} } , thus transforming the original, probably highly-dimensional, input to vectors in a K-dimensional space R K {\displaystyle \mathbb {R} ^{K}} .

Section 9

The standard logistic function is the special case for a 1-dimensional axis in 2-dimensional space, say the x-axis in the (x, y) plane. One variable is fixed at 0 (say z 2 = 0 {\displaystyle z_{2}=0}), so e 0 = 1 {\displaystyle e^{0}=1} , and the other variable can vary, denote it z 1 = x {\displaystyle z_{1}=x} , so e z 1 / ∑ k = 1 2 e z k = e x / (e x + 1) , {\textstyle e^{z_{1}}/\sum _{k=1}^{2}e^{z_{k}}=e^{x}/\left(e^{x}+1\right),} the standard logistic function, and e z 2 / ∑ k = 1 2 e z k = 1 / (e x + 1) , {\textstyle e^{z_{2}}/\sum _{k=1}^{2}e^{z_{k}}=1/\left(e^{x}+1\right),} its complement (meaning they add up to 1). The 1-dimensional input could alternatively be expressed as the line (x / 2 , − x / 2) {\displaystyle (x/2,\,-x/2)} , with outputs e x / 2 / (e x / 2 + e − x / 2) = e x / (e x + 1) {\displaystyle e^{x/2}/\left(e^{x/2}+e^{-x/2}\right)=e^{x}/\left(e^{x}+1\right)} and e − x / 2 / (e x / 2 + e − x / 2) = 1 / (e x + 1) . {\displaystyle e^{-x/2}/\left(e^{x/2}+e^{-x/2}\right)=1/\left(e^{x}+1\right).}